AULA VIVA

FEATHERSFORALL

Ignacio Ibañez - Juan Castillo - Iván Oliva -
Bastián González - Cristofer Jeria - Alex Flores

ENTREGABLES

Se debe producir: un contrato OpenAPI 3.1, endpoints RESTful, schemas reutilizables, reglas de calidad y documentación del ciclo de vida de la API, dejando definida la comunicación entre los componentes de FeathersForAll antes de la S06.

A CONTRATO API

- OpenAPI 3.1
- ⩾ 5 endpoints REST
- ⩾ 3 schemas reutilizables
- Seguridad JWT/Bearer

C CALIDA DE CONTRATO

- Spectral lint
- RFC 7807
- Idempotencia
- Ejemplos ejecutables

M INTEGRACIÓN

- Cliente TypeScript generado
- Política de versionado
- Mock server Prism
- Pull Request al repositorio

DECISIÓN

Estilo de API: REST + asíncrono

Estilo	¿Lo usamos?	Por qué
REST / HTTP JSON	Sí · API principal	Frontend HTML/JS en Vercel y API pública: cacheable, simple y con tooling maduro.
Async (202 + cola)	Sí · Tutor IA	Consultas al LLM encoladas en SQS (ADR 0003): la API no espera al modelo.
GraphQL	No	Un solo consumidor (nuestro frontend); no hay necesidades divergentes que lo justifiquen.
gRPC	No	Monolito modular: no hay microservicios internos que comunicar.
WebSocket / SSE	Más adelante	Posible para avisar cuando el tutor responde; en el MVP se consulta por GET.

CONTRATO

10 ENDPOINTS BAJO /V2

POST	/courses	Crear curso	H1
GET	/courses	Listar cursos (cursor)	H1
GET	/courses/{courseId}	Obtener curso	H1
POST	/courses/{courseId}/enrollments	Matricular estudiante	H1
POST	/courses/{courseId}/assessments	Crear evaluación	H2
PATCH	/assessments/{assessmentId}	Publicar o cerrar	H2
POST	/assessments/{assessmentId}/submissions	Rendir evaluación	H4
GET	/assessments/{assessmentId}/results	Dashboard de resultados	H3
POST	/courses/{courseId}/tutor-questions	Preguntar al tutor (202)	H5
GET	/tutor-questions/{questionId}	Respuesta del tutor	H5

DECISIÓN ARQUITECTÓNICA

SCHEMAS REUTILIZABLES Y TIPADOS

10
endpoints

15
schemas

7
respuestas de error

Course / CourseInput
Curso del tenant; periodo 2026-S2

Enrollment
Matrícula por correo institucional

Assessment + QuestionInput
Alternativas, correcta y feedback

SubmissionInput / Result
Respuestas → puntaje y feedback

AssessmentResults
Promedio y % de acierto por pregunta

TutorQuestion
queued → answered, grounded y fuentes

Problem (RFC 7807)
type, title, status, detail, trace_id

CoursePage
data + next_cursor (paginación)

DECISIÓN CLOUD

1. **Recursos, no verbos**
   /tutor-questions, no /askTutor

2. **Verbos con semántica**
   POST crea · GET lee · PATCH cambia estado

3. **Versionado explícito**
   v1 en todas las rutas; nunca se borra un campo

4. **Errores RFC 7807**
   application/problem+json con trace_id

5. **Idempotencia**
   Idempotency-Key (UUID) en todo POST, 24 h

6. **Paginación por cursor**
   cursor · limit · next_cursor

Ejemplo · 422 al publicar sin respuesta correcta


```
{
 "type": ".../errors/validation",
 "title": "Datos inválidos",
 "status": 422,
 "detail": "No se puede publicar: la
 pregunta 2 no tiene respuesta
 correcta.",
 "trace_id": "c8f2b1d4e7",
 "errors": [{
 "field": "questions[1].correct_option",
 "message": "Requerido para publicar."
 }]
}
```

MULTI-TENAT EN EL CONTRATO

JWT

El tenant viaja en el token

Bearer JWT del SSO con tenant_id y rol. Nunca en la URL.

RLS

El backend fija el tenant

SET LOCAL app.tenant_id por transacción (ADR 0002, D1).

404

Otro colegio no existe

Un recurso de otro tenant responde 404: no se revela que existe.

429

Rate limit por tenant

Retry-After y X-RateLimit-Remaining. Controla gasto de LLM (FinOps).

FLUJO ASÍNCRONO CON 202 ACCEPTED

Frontend

POST /courses/{id}/tutor-questions

Backend

202 + Location · encola

SQA

Mensaje con tenant y curso

Worker

RAG (pgvector) + LLM

Frontend

GET /tutor-questions/{id}

Porque 202 y no 200

El LLM puede tardar segundos; la API no bloquea sus hilos y soporta picos de pruebas. La caída del LLM no afecta cursos ni evaluaciones.

Respuesta TutorQuestion

status: queued → answered | failed
grounded: false ⇒ "no tengo información suficiente"
sources: solo apuntes del mismo curso y tenant

CONTRACT-FIRST

UN CONTRATO, CUATRO HERRAMIENTAS

api/openapi.yaml

Spectral

Lint con reglas del equipo

0 problemas

Prism

Mock server sin backend

5/5 ejemplos OK

openapi-typescript

Tipos del cliente TS

tsc sin errores

Swagger UI / Redoc

Documentación viva

editor.swagger.io

QUÉ CABE EN /V2 Y QUÉ EXIGE /V3

✓ Compatible · MINOR → /v2

- Agregar un endpoint
- Agregar campo opcional o de respuesta
- Agregar query param opcional
- Nuevo error documentado (RFC 7807)

! Rompe · MAJOR → /v3

- Eliminar o renombrar campo o ruta
- Cambiar tipo o formato
- Volver obligatorio un campo
- Cambiar auth o cómo se identifica el tenant

Deprecación: deprecated: true + headers Deprecation y Sunset · plazo mínimo de un semestre académico

HISTORIA → ENDPOINT → CLOUD

Historia de usuario	Endpoints	Cloud
H1 Crear cursos y matricular estudiantes	POST /courses · GET /courses · POST …/enrollments	Vercel → Render → PostgreSQL
H2 Crear evaluación auto-corregida	POST …/assessments · PATCH /assessments/{id}	Vercel → Render → PostgreSQL
H3 Ver dashboard de resultados	GET /assessments/{id}/results	Vercel → Render → PostgreSQL
H4 Rendir evaluación y recibir feedback	POST /assessments/{id}/submissions	Vercel → Render → PostgreSQL
H5 Consultar Tutor IA	POST …/tutor-questions · GET /tutor-questions/{id}	Rende → SQS → Worker → LLM

MUCHAS GRACIAS